

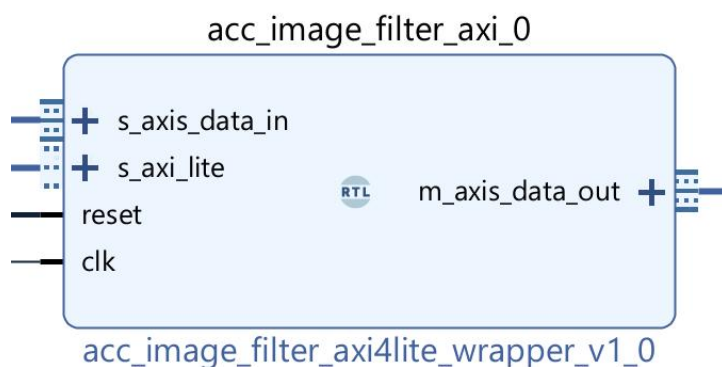
Digitalni VLSI sistemi

- Izveštaj za projekat -

Konstantin Simjanović 0013/2022
Vladimir Grijaković 0183/2022

Uputstvo za upotrebu hardverskog akceleratora za filtriranje slike

Jezgro akceleratora čini komponenta **acc_image_filter** koja vrši samu obradu slike, odnosno njeno filtriranje. Sama komponenta prima podatke i prosledjuje izlazne podatke (piksele) preko AXI4-Stream interfejsa ulazi u registre za kontrolu i konfiguraciju akceleratora nalaze si direktno na pinovima u ulazu komponente. Kako bi bilo nepraktično priljućivati u sistem komponentu sa tako mnogo pinova, u konfiguracione registre je moguće upisivati preko AXI4-Lite interfejsa. To omogućava omotač samog akceleratora **acc_image_filter_axi4lite_wrapper** vrši mu direktan pristup putem navedenih stream i lite interfejsa. Dakle u sistem je potrebno integrisati samo ovaj omotač, a on u sebi već sadrži akcelerator i obavlja sve potrebne pristupe. Omotač ima AXI4-Stream i AXI4-Lite interfejs, prvi za primanje piksela i slanje izlaznih odataka a drugi za konfiguraciju registara akceleratora. Pored toga ima signale reset i clk.



Registri dostupni za konfiguraciju su prikazani u tabeli ispod.

Adresa	Naziv registra	Opis
0x0	reg_ctrl	Kontrolni registar akceleratora. Sadržaj definisan u Tabeli 5.2.
0x2	reg_radius	Vrednost radijusa filtra. Filtrar ima jednaku visinu i širinu koja iznosi $2 \cdot \text{radius} + 1$. Najniža 3-bitna registra definišu vrednost radijusa iz opsega [0,4] ostali biti registra se ne koriste.
0x4	reg_img_w	Širina ulazne slike. Zadaće se kao 16 bitni neoznačeni ceo broj.
0x6	reg_img_h	Visina ulazne slike. Zadaće se kao 16 bitni neoznačeni ceo broj.
0x8	reg_coeff_scale	Faktor skaliranja koeficijenata. Ovim faktorom je potrebno skalirati izlazni rezultat filtriranja pre slanja na izlaz. Predstavlja neoznačeni 16-bitni podatak gde viša 4 bita predstavljaju ceo deo dok nižih 12 bita predstavlja razlomljeni deo faktora skaliranja.
0x10	reg_coeff_W0	Koeficijent W0 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.
0x12	reg_coeff_W1	Koeficijent W1 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.
...	...	...
0x170	reg_coeff_W80	Koeficijent W80 filtra zadat kao 16 bitni označeni broj sa 1 bitom za ceo deo i 15 bita za razlomljeni deo.

Prvi registar za kontrolu rada akceleratora podržava 2 moda - 0 i 1.

Mod 0 - akcelerator izbacuje 16bitne podatke, gornjih 8 bita je 0 a donjih 8 predstavlja neoznačen ceo broj u domenu 0-255.

Mod 1 - akcelerator izbacuje 16bitne podatke, gornjih 9 bita predstavlja ceo deo a donjih 7 razlomljeni. Dakle izlazni podatak je označen realan broj u formatu q9.7.

U gore datoj tabeli, pored registara nalaze se i njihove adrese za pristup putem AXI4-Lite interfejsa, a podaci koje akcelerator prima putem AXI4-Stream interfejsa su 8bitni neoznačeni celi brojevi (vrednosti piksela od 0 do 255).

Kako nemaju svi pikseli u slici dovoljno suseda za računanje izlaza, maska će biti primenjena samo na pikselima gde je to moguće, odnosno odredjen broj ivičnih piksela će biti preskočeno i izlazna slika će biti manja od ulazne. Takodje, kao što je već vidjeno, izlazni podaci su 2 puta veći od ulaznih. Ove stvari je potrebno imati na umu prilikom alokacije memorije potrebne sa skladištenje izlazne slike. Potrebna memorije je $(\text{visina ulazne slike} - 2 * (\text{redius filtra})) * (\text{širina ulazne slike} - 2 * (\text{redius filtra})) * 2$.

Akcelerator na izlazu ima zakašnjenje u odnosu na ulaz zbog punjenja unutrašnjih buffera i pipeline-a pa ce se tako na izlazu prvi izlazni podatak pojaviti tek nakon $2 * (\text{širina ulazne slike}) + 2 * (\text{radius filtra}) + 1 + 6$ taktova. Plava boja označava kašnjenje usled bunjenja buffera a crvena usled pipeline-a. Nakon ovog broja taktova, svaki takt izlazi novi izlazni piksel osim kada je na redu za obradu piksel sa nedovoljnim brojem suseda (ivični pikseli) pa se na kratko izbacuje broj nevalidnih izlaza.

Nakog integracije akceleratora (odnosno njegovog axi4lite omotača) u sistem i povezivanja sa DMA kontrolerom od koga će primati podatke i kome će slati podatke, potrebno ga je konfigurisati u softveru i pokrenuti njegov rad. Prvo je odredjenu strukturu potrebno upisati parametra akceleratora:

```
typedef struct ProcessingParams {  
    u16 ImgW;  
    u16 ImgH;  
    u16 FilterRadius;  
    u16 FilterCoeffsScale;  
    u16 FilterControl;  
    _Bool Border;  
    s16* FilterCoeffs;  
}ProcessingParams;
```

Za koeficijente filtra potrebno je alocirati dovoljno velik niz(u zavisnosti od radiusa, odnosno veličine maske) i kao pokazivač na njega postaviti gore prikazan s16* FilterCoeffs. Nije potrebno postaviti vrednosti svih 81 koeficijanata ukoliko se koristi manja maska (dakle za masku 3x3 alocira se red dužine 9).

Nakon izbora parametara i upisa u gore opisanu strukturu, potrebno je pozvati funkciju koja će izvršiti konfiguraciju dma kontrolera, upisa parametara i započeti slanje ulaznih piksela, a potom sačekati da akcelerator izvrši obradu cele slike.

```
static int ImageFilterHW(u8* DataBuffer, u16* ResultBuffer, ProcessingParams Params);
```

Argumenti ove funkcije su:

DataBuffer - pokazivač na početak memorije gde se nalazi ulazna slika

ResultBuffer - pokazivač na pomećat kemorije gde želimo da se izlazna slika upiše (voditi računa da se alocira dovoljno memorije, ranije obrazloženo)

Params - parametri za konfiguraciju registara akceleratora

Povratna vrednost funkcije tipa int može biti XST_FAILURE, ukoliko filtriranje nije uspešno odradjeno ili XST_SUCCESS, ukoliko je uspešno.

Implementacija akceleratora u VHDL-u

Implementacija hardverskog filtra podaljena je na nekoliko bitnih celina:

1. Brojači - korišćena su 2 brojača za praćenje kolone i reda, odnosno koordinate, piksela koji je poslednji primljen, odnosno poslednji validan piksel koji je ušao u sistem.

```
--proces brojaca ulaznih piksela
process(clk)
begin
    if rising_edge(clk) then
        if reset = '1' then
            kolona_count <= (others => '0');
            red_count <= (others => '0');
        else
            if (s_ready = '1' and s_valid = '1') then
                if kolona_count = (unsigned(reg_img_w) - 1) then
                    kolona_count <= (others => '0');
                    if red_count = (unsigned(reg_img_h) - 1) then
                        red_count <= (others => '0');
                    else
                        red_count <= red_count + 1;
                    end if;
                else
                    kolona_count <= kolona_count + 1;
                end if;
            end if;
        end if;
    end if;
end process;
```

2. Generisanje kontrolnih signala u AXI4-stream interfejsu - s_ready, m_valid i m_last.

s_ready - akcelerator je spreman da primi podatak

m_valid - podatak koji je akcelerator izvacio je validan

m_last - podatak koji je akcelerator izbacio je poslednji podatak, prenos se završava

```
--proces za generisanje s_axis_data_in_tready signala kojim se omogućava primanje piksela
process(reset, m_ready, m_valid) is
begin
    s_ready <= '0';

    if(reset = '1') then
        s_ready <= '0';
    else
        s_ready <= m_axis_data_out_tready or (not m_valid);
    end if;
end process;

--proces za generisanje m_valid i m_last signala; univerzalno za sve maske
process(reset, clk) is
begin
    if(reset = '1') then
        valid_pipeline <= (others => '0');
        last_pipeline <= (others => '0');
    elsif(rising_edge(clk)) then

        if(m_ready = '1' or m_valid = '0') then

            --generisanje m_last signala
            last_pipeline(1) <= last_pipeline(0);
            last_pipeline(2) <= last_pipeline(1);
            last_pipeline(3) <= last_pipeline(2);
            last_pipeline(4) <= last_pipeline(3);
            last_pipeline(5) <= last_pipeline(4);
            if (red_count = unsigned(reg_img_h) - 1 and kolona_count = unsigned(reg_img_w) - 1) and s_valid = '1' and s_ready = '1' then
                last_pipeline(0) <= '1'; -- Signaliziraj kraj na poslednjem pikselu slike
            else
                last_pipeline(0) <= '0';
            end if;

            --generisanje m_valid signala
            valid_pipeline(1) <= valid_pipeline(0);
            valid_pipeline(2) <= valid_pipeline(1);
            valid_pipeline(3) <= valid_pipeline(2);
            valid_pipeline(4) <= valid_pipeline(3);
            valid_pipeline(5) <= valid_pipeline(4);
            --provera da li je centralni piksel u kernelu ivicni ili ispada iz slike(u slucaju punjenjg ram bafera)
            if (red_count >= (2 * unsigned(reg_radius)) and red_count <= unsigned(reg_img_h) - 1 and
                kolona_count >= (2 * unsigned(reg_radius)) and kolona_count <= unsigned(reg_img_w) - 1) and s_valid = '1' and s_ready = '1' then
                valid_pipeline(0) <= '1';
            else
                valid_pipeline(0) <= '0';
            end if;

        end if;

    end if;
end process;
```

m_last će biti 1 samo kada nam na izlaz dodje piksel koji odgovara poslednjem ulaznom pikselu, a m_valid je 1 kada je na izlazu rezultat obrade piksela koji je imao dovoljno susedstvo za filtriranje.

3. Upis ulaznog piksela u ram_buffer, preuredjivanje piksela u ram_bufferu i upis piksela iz buffera u registre aritmetičke jedinice akceleratora

```
--proces upisa upisa piksela u ram buffer i iz ram buffera u registre aritmetičke jedinice; univerzalnoza sve maske
process(reset, clk) is
begin
    if(reset = '1') then
        shift_regs <= (others => (others => (others => '0')));
    elsif(rising_edge(clk)) then
        if(s_ready = '1' and s_valid = '1') then

            for i in 0 to 8 loop
                for j in 8 downto 1 loop
                    shift_regs(i)(j) <= shift_regs(i)(j-1);
                end loop;
            end loop;

            shift_regs(0)(0) <= s_data;
            shift_regs(1)(0) <= ram_buffer(TO_INTEGER(kolona_count))(7 downto 0);
            shift_regs(2)(0) <= ram_buffer(TO_INTEGER(kolona_count))(15 downto 8);
            shift_regs(3)(0) <= ram_buffer(TO_INTEGER(kolona_count))(23 downto 16);
            shift_regs(4)(0) <= ram_buffer(TO_INTEGER(kolona_count))(31 downto 24);
            shift_regs(5)(0) <= ram_buffer(TO_INTEGER(kolona_count))(39 downto 32);
            shift_regs(6)(0) <= ram_buffer(TO_INTEGER(kolona_count))(47 downto 40);
            shift_regs(7)(0) <= ram_buffer(TO_INTEGER(kolona_count))(55 downto 48);
            shift_regs(8)(0) <= ram_buffer(TO_INTEGER(kolona_count))(63 downto 56);

            ram_buffer(TO_INTEGER(kolona_count)) <= ram_buffer(TO_INTEGER(kolona_count))(55 downto 0) & s_data;

        end if;
    end if;
end process;
```

4. Proces obrade podataka (filtriranje) u aritmetičkoj jedinici akceleratora; prikaz samo najbitnijih delova, 4 pipeline nivoa:

Prvi nivo - množenje piksela sa koeficijetima

```
--prvi pipeline nivo - množenje sa koeficijetima
for i in 0 to 8 loop
    for j in 0 to 8 loop
        if (i < v_dim and j < v_dim) then
            --odredjivanje koeficijenta sa kojim se mnozi trenutni piksel
            v_coeff_idx := (v_dim * v_dim - 1) - (i * v_dim + j);
            data_proizvodi(i)(j) <= signed('0' & shift_regs(i)(j)) * signed(reg_coeff(v_coeff_idx));
        else
            -- nevalidni pikseli se mnoze sa 0
            data_proizvodi(i)(j) <= (others => '0');
        end if;
    end loop;
end loop;
```

Drugi nivo - zbir svih tih proizvoda po redovima


```

--drugi pipeline nivo - zbir proizvoda po redovima
for i in 0 to 8 loop
    v_suma := (others => '0');
    for j in 0 to 8 loop
        v_suma := v_suma + resize(data_proizvodi(i)(j), 28);
    end loop;
    data_zbir_red(i) <= v_suma; -- Rezultat se upisuje u registar na kraju takta
end loop;

```

Treći nivo - zbir svih prethodno dobijenih suma

```

--treci pipeline nivo - zbir svih suma redova
data_zbir <= resize(data_zbir_red(0), 32) + data_zbir_red(1) + data_zbir_red(2) +
    data_zbir_red(3) + data_zbir_red(4) + data_zbir_red(5) +
    data_zbir_red(6) + data_zbir_red(7) + data_zbir_red(8);

```

Četvrti nivo - skaliranje sa coeff_scale

```

--cetvrti pipeline stepen - mnozenje sa coeff_scale
data_izlaz <= data_zbir * signed(reg_coeff_scale);

```

Kao što se vidi, konačan rezultat aritmetičke jedinice se nalazi u data_izlaz registru, međutim ta vrednost nije spremna za slanje na izlaz jer nije dobro formatirana - koeficijenti sa kojima smo množili nisu celobrojni, takodje postoje 2 moda za izlazne podatke pa nam je potreban još jedan blok za procesiranje ovog rezultata kako bi bio spreman za izlaz:

5. Formatiranje izlaznog podatka

```

--process za formatiranje izlaznog podatka
process(clk)
    variable data_shifted : signed(47 downto 0);
begin
    if rising_edge(clk) then
        if reset = '1' then
            m_data <= (others => '0');
        else
            if(m_ready = '1' or m_valid = '0') then
                if reg_ctrl(0) = '0' then --izlazni podatak 8bitni neoznaceni ceo broj

                    --jer smo množili sa ne-celim brojevima coeff i coeff_scale
                    data_shifted := shift_right(data_izlaz, 27);

                    --zakocavanje na maksimalnu odnosno minimalnu vrenost ukoliko je to potrebno
                    if (data_shifted > 255) then
                        m_data <= x"00FF";
                    elsif (data_shifted < 0) then
                        m_data <= x"0000";
                    else
                        m_data <= x"00" & std_logic_vector(data_shifted(7 downto 0));
                    end if;

                else
                    --pomeranje za 20 u desno kako bi ostalo 7 bita za necelobrojni deo
                    data_shifted := shift_right(data_izlaz, 20);
                    m_data <= std_logic_vector(resize(data_shifted, 16));
                end if;
            end if;
        end if;
    end if;
end process;

```

Koeficijenti kojima smo množili ulazne piksele su u fromatu q1.15, a coeff_scale je u fromatu q4.12. To znači da je rezultat koji smo dobili kao izlaz aritmetičke jedinice akceleratora takav da mu je donjih 27 bita razlomljeni deo. U modu 0 gde želimo ceo broj kao izlaz zato radimo shift u desno za 27 kako bi nam samo celobrojni deo i ostao a u modu 1 radimo za 20 kako bi nam ostalo 7 bita za razlomljeni deo.

Implementacija softverskih funkcija za inicijalizaciju i pokretanje akceleratora

Deklaracija funkcije za korišćenje akceleratora je već pokazana pa sad sledi kako je ona implementirana. Slično kao i hardver, ovu funkciju možemo rasčlaniti na više delova.

1. Inicijalizacija i konfiguracija DMA kontrolera

```
XAxiDma_Config *AxiDmaConfigPtr;
AxiDmaConfigPtr = XAxiDma_LookupConfig(BaseAddress: XPAR_XAXIDMA_0_BASEADDR);
if (!AxiDmaConfigPtr) {
    xil_printf(ctr1: " HW CONFIG ERROR: No config found for %d\r\n", XPAR_XAXIDMA_0_BASEADDR);

    return XST_FAILURE;
}

Status = DmaConfigure(AxiDmaConfigPtr, AxiDmaPtr: &AxiDma);
if (Status != XST_SUCCESS)
{
    xil_printf(ctr1: " HW CONFIG ERROR: DMA configuration failed\r\n");
    return XST_FAILURE;
}
```

Kao što se vidi za konfiguraciju je korišćena funkcija DmaConfigure koja izgleda ovako:

```
static int DmaConfigure(XAxiDma_Config* AxiDmaConfigPtr, XAxiDma* AxiDmaPtr)
{
    int Status;

    /* DMA configuration */

    Status = XAxiDma_CfgInitialize(InstancePtr: AxiDmaPtr, Config: AxiDmaConfigPtr);
    if (Status != XST_SUCCESS) {
        xil_printf(ctr1: " ERROR: DMA initialization failed %d\r\n", Status);
        return XST_FAILURE;
    }

    if (XAxiDma_HasSg(AxiDmaPtr)) {
        xil_printf(ctr1: " ERROR: DMA configure in SG mode \r\n");
        return XST_FAILURE;
    }

    /* Configure DMA interrupts */
    Status = XSetupInterruptSystem(DriverInstance: AxiDmaPtr, IntrHandler: &TxIntrHandler,
                                  IntrId: AxiDmaConfigPtr->IntrId[0], IntcParent: AxiDmaConfigPtr->IntrParent,
                                  Priority: XINTERRUPT_DEFAULT_PRIORITY);
    if (Status != XST_SUCCESS) {
        xil_printf(ctr1: " ERROR: Cannot configure DMA TX interrupt\r\n");
        return XST_FAILURE;
    }

    Status = XSetupInterruptSystem(DriverInstance: AxiDmaPtr, IntrHandler: &RxIntrHandler,
                                  IntrId: AxiDmaConfigPtr->IntrId[1], IntcParent: AxiDmaConfigPtr->IntrParent,
                                  Priority: XINTERRUPT_DEFAULT_PRIORITY);
    if (Status != XST_SUCCESS) {
        xil_printf(ctr1: " ERROR: Cannot configure DMA RX interrupt\r\n");
        return XST_FAILURE;
    }

    XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DMA_TO_DEVICE);
    XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DEVICE_TO_DMA);

    return XST_SUCCESS;
}
```


2. Konfigurisanje akceleratora - upis parametara u konfiguracione registre akceleratora preko AXI4-Lite interfejsa

```
Status = AccConfigure(BaseAddress: XPAR_ACC_IMAGE_FILTER_AXI4LITE_WRAPPER_0_BASEADDR, Params);
```

Ova funkcija samo uzima parametre prethodno upakovane u strukturu koja je vec pokazana i upisuje preko AXI4-Lite interfejsa na adrese koje su takodje ranije pokazane. Primer upisa u jedan registar:

```
Xil_Out16(Addr: BaseAddress + REG_CTRL, Value: Params.FilterControl);

ReadValue = Xil_In16(Addr: BaseAddress + REG_CTRL);
if(ReadValue != Params.FilterControl)
    return XST_FAILURE;
```

3. Pokretanje prenosa podatak i čekanje da se prenos podataka i prijem novih od akceleratora završi

```
Status = DmaStartTransfers(AxiDmaPtr: &AxiDma, TxBuffer: DataBuffer, TxSize: ImageSizeIn, RxBuffer: (u8*)ResultBuffer, RxSize: ImageSizeOut);
if (Status != XST_SUCCESS)
{
    xil_printf(ctrl1: " HW CONFIG ERROR: Starting DMA transfers failed\r\n");
    return XST_FAILURE;
}

Status = DmaWaitTransfers(TxFlag: &TxDone, RxFlag: &RxDone, Timeout: DMA_TRANSFER_TIMEOUT);
if (Status != XST_SUCCESS)
{
    xil_printf(ctrl1: " HW PROC ERROR: Completing DMA transfers failed\r\n");
    return XST_FAILURE;
}
```

4. Kraj obrade - blokiranje prekida od DMA kontrolera i njegovo restovanje kako bi mogao opet da bude upotrebljen u sledećoj obradi slike

```
/* Disable TX and RX interrupts */
XDisconnectInterruptCntl(IntrId: AxiDmaConfigPtr->IntrId[0], IntcParent: AxiDmaConfigPtr->IntrParent);
XDisconnectInterruptCntl(IntrId: AxiDmaConfigPtr->IntrId[1], IntcParent: AxiDmaConfigPtr->IntrParent);

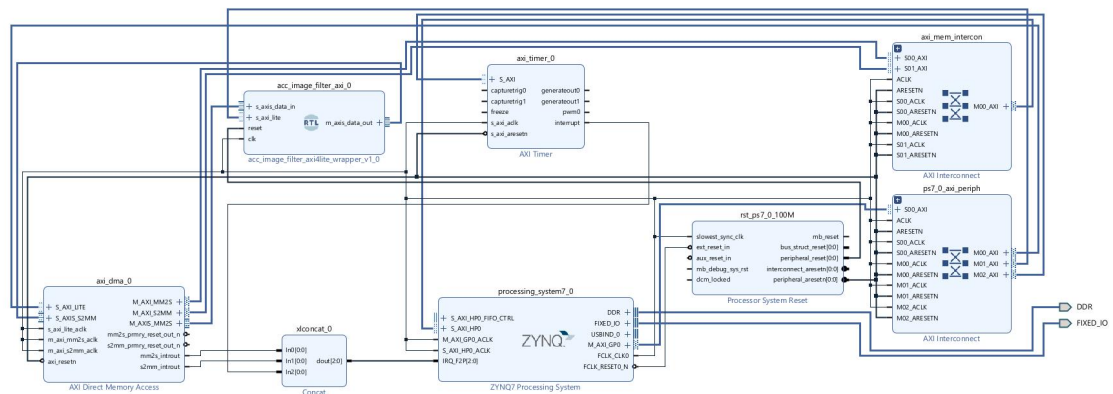
/* DODATO: Resetuj DMA hardver pre izlaska iz funkcije */
XAxiDma_Reset(InstancePtr: &AxiDma);

/* Opciono: Sačekaj kratko da se reset završi */
int ResetTimeout = 10000;
while (ResetTimeout > 0) {
    if (XAxiDma_ResetIsDone(InstancePtr: &AxiDma)) {
        break;
    }
    ResetTimeout--;
}

return XST_SUCCESS;
```

Testiranje, rezultati i performanse

Pored hardverskog akceleratora (i dma kontrolera koji komunicira sa akceleratorom), u sistemu je korišćen i tajmer za merenje vremena izvršavanja funkcija i njihovo upoređivanje. Blok dijagram celog sistema:



Što se softvera tiče, osim prethodno pokazane funkcije za rad sa hardverskom obradom, korišćene su i funkcija za softversku obradu kao i funkcija za poredjenje softverske i hardverske obrade.

```
static void ImageFilterSW(u8* DataBuffer, u8* ResultBuffer, ProcessingParams Params);
static int CheckData(u16* ResultBuffer, u8* ReferentBuffer, ProcessingParams Params);
```

Slede primeri filtriranja sa različitim parametrima, ulazna slika je za sve primere ista:



Primer 1: Ulazna slika 128x128, box filtar 3x3, mod 0

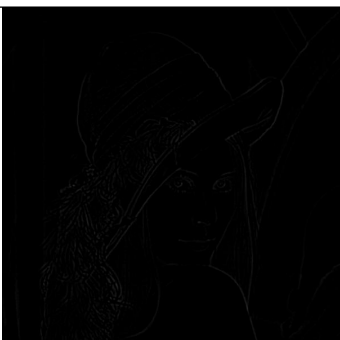
<pre>// Define processing parameters Params.ImgH = 128;Params.ImgW = 128; Params.FilterControl = 0x0000; Params.FilterRadius = 1; s16 Coeffs[] = { 3641, 3641, 3641, 3641, 3641, 3641, 3641, 3641, 3641 }; Params.FilterCoeffs = Coeffs; Params.FilterCoeffsScale = 4096;</pre>	
---	--

Vreme softverske obrade: 39,7 ms

Vreme hardverske obrade: 5,7 ms

Koliko puta je brža hardverska obrada: 7

Primer 2: Ulazna slika 512x512, LoG filtar 7x7, mod 1

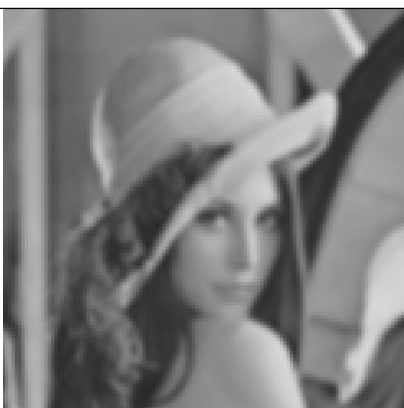
<pre>// Define processing parameters Params.ImgH = 512;Params.ImgW = 512; Params.FilterControl = 0x0001; Params.FilterRadius = 3; Params.FilterCoeffsScale = 4096; s16 CoeffsLoG7x7[] = { 10, 86, 281, 406, 281, 86, 10, 86, 573, 1284, 1412, 1284, 573, 86, 281, 1284, 0, -3163, 0, 1284, 281, 406, 1412, -3163, -10430, -3163, 1412, 406, 281, 1284, 0, -3163, 0, 1284, 281, 86, 573, 1284, 1412, 1284, 573, 86, 10, 86, 281, 406, 281, 86, 10 }; Params.FilterCoeffs = CoeffsLoG7x7;</pre>	
---	---

Vreme softverske obrade: 3,11 s

Vreme hardverske obrade: 7,8 ms

Koliko puta je brža hardverska obrada: 395

Primer 3: Ulazna slika 128x128, gauss 5x5, mod 1

<pre>// Define processing parameters Params.ImgH = 128;Params.ImgW = 128; Params.FilterControl = 0x0001; Params.FilterRadius = 2; Params.FilterCoeffsScale = 4096; s16 CoeffsGauss5x5[] = { 98, 436, 718, 436, 98, 436, 1953, 3221, 1953, 436, 718, 3221, 5312, 3221, 718, 436, 1953, 3221, 1953, 436, 98, 436, 718, 436, 98 }; Params.FilterCoeffs = CoeffsGauss5x5;</pre>	
---	--

Vreme softverske obrade: 99,4 ms

Vreme hardverske obrade: 5,7 ms

Koliko puta je brža hardverska obrada: 17

Primer 4: Ulazna slika 512x512, sharp box 9x9, mod 0

```
// Define processing parameters
Params.ImgH = 512;Params.ImgW = 512;
Params.FilterControl = 0x0000;
Params.FilterRadius = 4;
Params.FilterCoeffsScale = 8192; // Faktor skaliranja 2

// Matrica 9x9 (81 element)
// Svi elementi su -202, osim centralnog koji je 32566
s16 CoeffsSharpBox9x9[81];
for(int i = 0; i < 81; i++) {
    CoeffsSharpBox9x9[i] = -202;
}
CoeffsSharpBox9x9[40] = 32566; // Centralni element (indeks 40)

Params.FilterCoeffs = CoeffsSharpBox9x9;
```



Vreme softverske obrade: 5 s

Vreme hardverske obrade: 7,8 ms

Koliko puta je brža hardverska obrada: 639

Primer 5: Ulazna slika 128x128, sharp gauss 5x5, mod 0

```
// Define processing parameters
Params.ImgH = 128;Params.ImgW = 128;
Params.FilterControl = 0x0000;
Params.FilterRadius = 2;
Params.FilterCoeffsScale = 8192; // Faktor skaliranja 2
s16 CoeffsSharpGauss5x5[] = {
    -49,  -218,  -359,  -218,  -49,
    -218,  -977, -1611,  -977,  -218,
    -359, -1611, 30112, -1611,  -359,
    -218,  -977, -1611,  -977,  -218,
    -49,  -218,  -359,  -218,  -49
};
Params.FilterCoeffs = CoeffsSharpGauss5x5;
```



Vreme softverske obrade: 99,4 ms

Vreme hardverske obrade: 5,7 ms

Koliko puta je brža hardverska obrada: 17

Kao što se vidi, vreme obrade slike u hardveru, pomoću akceleratora je drastično kraće od softverske obrade, što je veća slika i što je veći radius filtra to je razlika veća. Za sliku veličine 512x512 i radius 4 (prozor 9x9) obrada u hardveru je brža 639 puta.